

Objects, Classes & Inheritance

Post-Assessment 4 · 8 questions · 29 points

Name: _____

Date: _____

Score: / 29

This is a **post-assessment**. It shows what you learned in this unit. Try every question, and check your answers if you have time.

Work on your own, without a computer. You have about **40 minutes**. When a question asks what a program prints, write exactly what Python would print, one line of output per answer line. If the code would crash, write the name of the error (for example, `TypeError`). Show your thinking; partial credit is given.

1. Objects everywhere

3 points

(a) What does `print(type({"a": 1}))` show? (1 pt)

(b) Which built-in function shows the documentation for an object? (1 pt)

(c) In `"hi".upper()`, `"hi"` is an object of the `str` class. What is `upper` called? (1 pt)

2. Robot class

4 points

```
class Robot:
    def __init__(self, name):
        self.name = name
        self.battery = 10

    def walk(self, steps):
        self.battery = self.battery - steps
        return f"{self.name} walked {steps} steps"

    def charge(self):
        self.battery = 10

r1 = Robot("Bolt")
r2 = Robot("Zip")
print(r1.walk(3))
r2.walk(4)
r1.walk(2)
r2.charge()
print(r1.battery, r2.battery)
```

(a) Write exactly what the program prints.

(2 pts)

(b) When `r2.walk(4)` runs, what does `self` refer to?

(1 pt)

(c) How many `Robot` objects (instances) does the program create?

(1 pt)

3. Readable objects

2 points

```
class Player:
    def __init__(self, name, score):
        self.name = name
        self.score = score

    def __str__(self):
        return f"{self.name}: {self.score} pts"

print(Player("Kai", 40))
```

(a) What does this print?

(1 pt)

(b) What would `print` show if the class had **no** `__str__` method?

(1 pt)

4. Zoo Manager

4 points

```
class Exhibit:
    def __init__(self, animal, count):
        self.animal = animal
        self.count = count

class Zoo:
    visitors = 0

    def __init__(self, name):
        self.name = name
        self.exhibits = []

    def add(self, exhibit):
        self.exhibits.append(exhibit)

    def total_animals(self):
        total = 0
        for e in self.exhibits:
            total = total + e.count
        return total

z = Zoo("City Zoo")
z.add(Exhibit("lions", 3))
z.add(Exhibit("penguins", 12))
z.add(Exhibit("otters", 5))
print(len(z.exhibits), z.exhibits[1].animal)
print(z.total_animals())
print(z.exhibits[-1].count * 2)
```

(a) Write exactly what the program prints.

(3 pts)

(b) Is `visitors` a class attribute or an instance attribute? What does that mean?

(1 pt)

5. Vehicle family

4 points

```
class Vehicle:
    def __init__(self, name):
        self.name = name
        self.wheels = 4

    def describe(self):
        return f"{self.name} has {self.wheels} wheels"

class Bike(Vehicle):
    def __init__(self, name, color):
        super().__init__(name)
        self.wheels = 2
        self.color = color

class Boat(Vehicle):
    def describe(self):
        return f"{self.name} floats"

rides = [Boat("Nemo"), Vehicle("Van"), Bike("Zoom", "red")]
for r in rides:
    print(r.describe())
print(rides[2].color)
```

Write exactly what the program prints.

6. isinstance()

3 points

Use the `rides` list from Question 5. Write **True** or **False**.

Expression	Result
<code>isinstance(rides[2], Bike)</code>	
<code>isinstance(rides[0], Vehicle)</code>	
<code>isinstance(rides[1], Boat)</code>	

7. Bug Bounty

3 points

This class has **two** bugs.

```
class Scoreboard:
    def __init__(self):
        self.points = 0

    def add(self, amount):
        self.points + amount

    def show():
        print("Points:", self.points)

s = Scoreboard()
s.add(4)
s.add(6)
s.show()
```

(a) Find and fix both bugs. Write the corrected lines.

(2 pts)

(b) After your fixes, what does the program print?

(1 pt)

8. Virtual Pet Care

6 points

```
class Pet:
    def __init__(self, name):
        self.name = name
        self.hunger = 5

    def feed(self):
        self.hunger = self.hunger - 1
```

Write a class `Puppy` that **inherits** from `Pet`:

- `__init__(self, name, toy)` uses `super()` to set the name and hunger, and also saves `toy`
- it **overrides** `feed()` so hunger goes down by 2 instead of 1
- it has a `__str__` method so `print(Puppy("Biscuit", "ball"))` shows `Biscuit (hunger 5)`

END OF ASSESSMENT